

04

2시간

CLAUDE.md와 룰 만들기

프로젝트 메모리

범위 설정

참조 링크

동적 컨텍스트

환경 설정

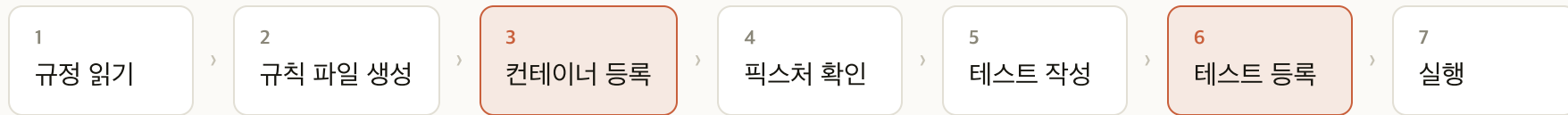
Hook

이 챕터에서 다루는 내용

주제	내용	실습
프로젝트 메모리	기술 스택, 폴더 구조, 코딩 컨벤션을 파일로 남기기	1
범위 설정	전역과 프로젝트, 두 곳에 나눠 두는 기준	3
고급 설정	참조 링크, 금지 규칙, 예제 코드, 동적 컨텍스트	3 · 4
환경 설정	권한 규칙의 범위, 환경변수	4
Hook	세션 상태 주입, 잘못된 도구 호출 차단	4
규칙 아홉 개	R002부터 R010까지. 마지막 하나는 다건 규칙	2 · 5 · 6

규칙을 아홉 개 더 만드는 것이 이 챕터의 실습입니다.
 같은 절차를 아홉 번 되풀이하므로, 그 절차를 먼저 파일로 정리한 뒤에 시작합니다.

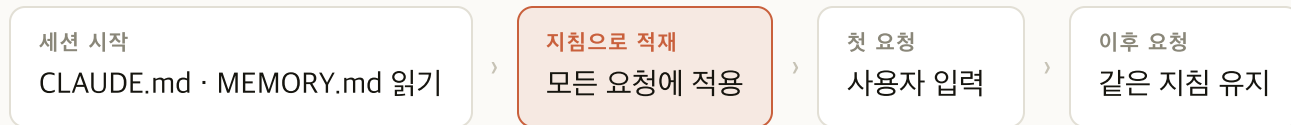
규칙 하나를 만드는 일곱 단계



- **매번 설명하는 부분:** 파일을 어디에 두는지, 어느 클래스를 상속하는지, 이름을 어떻게 붙이는지입니다. 모두 프로젝트가 이미 정해 둔 사항입니다.
- **등록할 두 곳:** 작성한 규칙을 앱과 테스트의 컨테이너(`container.py`)에 모두 추가합니다. 컨테이너는 규칙을 조립하는 코드를 담는 파일입니다.
 - 개별 테스트는 규칙 파일을 직접 지정해 실행하므로 컨테이너에 규칙이 빠져 있어도 통과합니다. 따라서 테스트가 통과했더라도 `container.py`에 해당 규칙을 추가했는지 확인해야 합니다.

매번 설명하던 부분은 `CLAUDE.md`에 적고, 등록 누락은 Hook으로 막습니다.
설명은 파일에 한 번만 적고, 누락은 사람이 아니라 스크립트가 잡습니다.

메모리의 적재 시점



- **세션 시작 시 자동 적재:** 파일을 읽어 달라고 하지 않아도 세션이 열릴 때 이미 적재됩니다.
- **/clear 이후에도 유지:** /clear 로 대화를 비워도 지침은 그대로 남습니다. 챕터 3에서 본 컨텍스트 점유 내역의 고정 항목입니다.
- **길이에 비례하는 비용:** 매 요청에 함께 실리므로 파일이 길수록 토큰을 더 씁니다. CLAUDE.md 는 전문이 들어가고, MEMORY.md 는 앞 200 줄까지만 들어갑니다.

대화는 그 세션에서 끝나고, 파일은 다음 세션에도 적용됩니다.
대화로 알려 준 것과 파일로 적어 둔 것의 차이입니다.

메모리 파일의 위치와 범위

위치	적용 범위	저장소	공유	적는 내용
~/.claude/CLAUDE.md	사용자의 모든 프로젝트	저장소 밖	개인 전용	응답 언어, 개인 작업 습관
./CLAUDE.md	이 프로젝트	커밋됨	팀 전체	기술 스택, 폴더 구조, 코딩 컨벤션
./CLAUDE.local.md	이 프로젝트	.gitignore 로 제외	개인 전용	로컬 경로, 개인 실험 메모
하위 폴더/CLAUDE.md	그 폴더 아래	커밋됨	팀 전체	그 폴더에만 해당하는 규약
~/.claude/projects/.../MEMORY.md	이 프로젝트	저장소 밖	개인 전용	Claude가 알아낸 것과 개인용으로 남긴 메모

가까운 파일이 먼 파일을 덮어쓰지 않습니다. 두 파일이 함께 적용됩니다.

따라서 팀이 함께 지킬 내용은 프로젝트 파일에, 개인에게만 해당하는 내용은 전역 파일이나 `CLAUDE.local.md` 에 나눠 둡니다.

프로젝트 메모리의 세 항목

항목	적는 내용	정하는 주체
기술 스택	Python 3.12, uv, pytest. 실행 명령은 <code>uv run</code> 으로 시작	코드에 이미 있음
폴더 구조	계층별 폴더의 역할, 규칙 파일과 테스트 파일의 위치	코드에 이미 있음
코딩 컨벤션	파일 이름 규칙, 클래스 상속 대상, 등록 지점, 로그에 남기지 않을 항목	사람이 정해야 함

- **적지 않는 것:** 코드를 읽으면 바로 아는 사실, 한 번 쓰고 마는 맥락, 지금 작업의 진행 상황.

앞의 두 항목은 `/init` 이 코드를 훑어 채웁니다.

코딩 컨벤션은 코드 어디에도 없는 약속이므로 사람이 적어 넣어야 합니다. 실습 1에서 직접 적는 부분입니다.

메모리를 남기는 세 방법

PowerShell · Claude Code 세션 안에서 입력

```
# 1. 저장소를 훑어 CLAUDE.md 를 새로 만들
> /init

# 2. 앞에 #을 붙이면 CLAUDE.md 나 MEMORY.md 에 바로 저장. 위치를 묻지 않음
> #규칙 파일의 이름은 반드시 영어로 작성합니다

# 3. CLAUDE.md 와 연결된 문서 목록을 열어 직접 고침
> /memory
```

/init 은 한 번, **#** 은 수시로 씁니다.

같은 설명을 두 번 하면 그 자리에서 **#** 으로 남깁니다. 세 번째부터는 설명하지 않아도 됩니다.

실습 1 · 프로젝트 메모리 작성

- 1 C:\Users\Administrator\lab 에서 `claude` 실행 후 `/init` 입력
- 2 생성된 `CLAUDE.md` 를 열어 기술 스택과 폴더 구조가 맞게 적혔는지 확인
- 3 코딩 컨벤션 추가 요청: 규칙 파일의 이름 규약, 상속할 클래스, code 와 name 의 출처, 테스트와 픽스처 위치, 등록할 두 곳을 `CLAUDE.md` 의 코딩 컨벤션에 추가해 줘
- 4 금지 항목 추가 요청: 카드번호, 계좌번호, 주민등록번호는 로그나 출력에 남기지 않는다는 항목을 `CLAUDE.md` 의 금지 절에 추가해 줘
- 5 `#규칙` 파일의 이름은 반드시 영어로 작성합니다 입력 후 저장 안내에서 어느 파일에 저장되었는지 확인
- 6 `/memory` 로 `CLAUDE.md` 를 열어 앞의 네 항목 확인
- 7 임시 지침 요청: 터미널 폭이 좁으니 표는 80칸 안에서 출력해 줘. 저장소에 남길 규약은 아니니 `MEMORY.md` 에 적어 줘
- 8 메모리 확인 요청: `MEMORY.md` 에 무엇이 들어 있는지 보여 줘

3번은 챕터 3에서 직접 해 본 일을 문장으로 옮기는 것입니다.

R001을 만들며 파일 위치와 상속 대상, 등록 지점을 이미 정해 보았습니다. 그 경험이 있어야 무엇을 적을지 판단할 수 있습니다.

/init 이 채운 부분

CLAUDE.md · 저장소를 훑어 만든 초안

기술 스택

- Python 3.12 이상을 씁니다.
- uv 로 패키지와 가상환경을 관리합니다. 실행은 `uv run` 으로 시작합니다.
- pytest. `pythonpath` 는 `src` 입니다.

폴더 구조

src/core/domain/	모델과 규칙. 바깥 계층을 모릅니다
src/core/service/	입력 · 처리 · 출력. 계층마다 base.py 에 추상 클래스가 있습니다
src/application/	조립과 진입점

/init 은 저장소를 훑어 초안을 만듭니다.

pyproject.toml 과 폴더 이름에 이미 있는 내용을 문장으로 옮긴 것입니다.

절차를 문장으로 옮기는 기준

적을 것		적지 않을 것	
위치	규칙은 <code>rules/</code> , 테스트는 <code>tests/rules/</code>	판정 조건	규정 문서에 이미 있음
이름	<code>rXXX_설명.py</code> , <code>test_rXXX.py</code>	위반 문구	규정 문서에 이미 있음
상속	단건은 <code>RowRule</code> , 다건은 <code>BatchRule</code>	진행 상황	고칠 때마다 어긋남
등록	컨테이너 두 곳에 모두		

결정이 끝난 것만 적습니다.

규정에 있는 내용을 `CLAUDE.md` 에 옮겨 두면 규정이 바뀔 때 두 곳이 어긋납니다. 어디를 보면 되는지만 남깁니다.

문장으로 고정한 절차

CLAUDE.md · 요청 두 번으로 채운 코딩 컨벤션과 금지

코딩 컨벤션

규칙 추가

- 규칙 파일은 `src/core/domain/rules/rXXX\_영문설명.py` 로 만듭니다.
- 단건 규칙은 `RowRule`, 다건 규칙은 `BatchRule` 을 상속합니다.
어느 쪽인지는 규정 항목 제목에 적혀 있습니다.
- `code` 와 `name` 은 규정 항목 제목에서 그대로 가져옵니다.
- 테스트는 `tests/rules/test\_rXXX.py` 에 만들고
픽스처 `tests/rules/fixtures/rXXX.csv` 를 씁니다.
- **등록은 두 곳입니다.** `src/application/container.py` 의 `build\_service` 와
`tests/container.py` 의 `all\_rules` 입니다.

금지

- 카드번호, 계좌번호, 주민등록번호를 로그나 출력에 남기지 않습니다.

실습 2 · R002로 메모리 확인

- 1 `/clear` 로 대화 비우기: 앞의 설명이 남지 않은 상태에서 확인
- 2 규칙 생성 요청: 경비규정 R002 를 구현해 줘
- 3 만들어진 파일의 위치와 이름, 상속한 클래스를 확인
- 4 컨테이너 두 곳의 등록 확인
- 5 `uv run pytest` 로 `2 passed` 확인

요청은 한 줄뿐입니다.

위치도 이름도 상속할 클래스도 말하지 않았습니다. 그 내용이 `CLAUDE.md` 에 있고, 판정 조건은 규정 문서에 있습니다.

챕터 3과 챕터 4의 요청 비교

	챕터 3 · R001	챕터 4 · R002
요청	규정을 읽고 규칙을 만들어 줘. 파일은 rules 폴더에, RowRule 을 상속해서, 컨테이너 두 곳에 등록해 줘	경비규정 R002 를 구현해 줘
만들어진 것	규칙 · 테스트 · 등록	규칙 · 테스트 · 등록
테스트	1 passed	2 passed

결과는 같고 요청만 짧아졌습니다.

빠진 설명이 CLAUDE.md 로 옮겨 갔을 뿐입니다. 규칙을 여덟 개 더 만들 것이므로 이 차이가 여덟 번 반복됩니다.

문서 참조의 두 방법

CLAUDE.md · 문서를 가리키는 두 표기

@ 를 붙이면 그 파일의 내용이 CLAUDE.md 뒤에 그대로 이어짐
@docs/경비규정.md

경로만 적으면 내용이 붙지 않고, 필요할 때 모델이 그 파일을 읽음
데이터 속성 이름은 `docs/데이터-스키마.md`에 있습니다.

- **두 표기의 공통점:** 내용을 CLAUDE.md에 옮겨 적지 않습니다. 원본이 한 곳에만 있으므로 문서가 바뀌면 그 문서만 고치면 됩니다.
- **두 표기의 차이:** @로 붙인 것은 매 요청에 함께 실리고, 경로만 적은 것은 실리지 않습니다.

매번 필요한 것만 @로 붙입니다.

규정 문서는 규칙을 만들 때마다 봐야 하므로 붙이고, 가끔 보는 문서는 경로만 적어 둡니다.

금지 규칙의 두 층

	CLAUDE.md 문장	permissions.deny
성격	지침	차단
어긴 경우	그대로 진행됨	도구 호출이 거부됨
적용 대상	판단이 필요한 일	대상이 분명한 일
예	카드번호를 로그에 남기지 않습니다	<code>Edit(tests/golden/**)</code>

막을 수 있는 것은 막고, 나머지를 문장으로 적습니다.

챕터 3에서 골든 데이터를 `deny` 로 잠근 것이 오른쪽입니다. 경로로 지목할 수 없는 약속은 왼쪽에 남을 수밖에 없습니다.

예제 코드 지정

CLAUDE.md · 설명 대신 이미 만든 파일 하나를 본보기로 지목

```
# 방법 1. 파일을 그대로 지목
새 규칙은 @src/core/domain/rules/r001_required_fields.py 의 구조를 따릅니다.

# 방법 2. 짧은 뼈대만 직접 작성
class R002(RowRule):
    code = "R002"
    name = "금액 유효성"
    def check(self, expense): ...
```

파일을 그대로 지목하는 편이 구조를 문장으로 설명하는 것보다 짧고 정확합니다.

속성 이름과 메서드 이름을 글로 적으면 길어지고, 코드가 바뀌어도 그 설명은 그대로 남습니다. 방법 1은 파일이 바뀌면 예제도 함께 바뀝니다.

실습 3 · 프로젝트 메모리 보강

- 1 참조 링크 추가: CLAUDE.md 에서 경비규정 문서는 @ 로 붙이고 데이터 스키마 문서는 경로만 적도록 고쳐 줘
- 2 예제 지목 추가: R001 파일을 본보기로 가리키는 문장을 CLAUDE.md 에 넣어 줘
- 3 전역 파일 만들기: /memory 에서 User instructions 를 골라 답변은 한국어로 합니다 한 줄 추가
- 4 /memory 를 다시 열어 아래 네 줄이 모두 보이는지 확인
- 5 /context 로 메모리 파일이 차지하는 양 확인

/memory

```
Memory
Auto-memory: on

1. Project instructions
    Checked in at ./CLAUDE.md
2. L src/core/domain/rules/
    r001_required_fields.py    @-imported
3. L docs/경비규정.md          @-imported
4. User instructions
    Saved in ~/.claude/CLAUDE.md
5. Open auto-memory folder
```

1번과 4번이 함께 보이면 두 범위가 모두 적용된 것입니다.

@ 로 붙인 문서는 2·3번처럼 하위에 함께 표시되고, 경로만 적은 스키마 문서는 목록에 없습니다. 4번 파일은 저장소가 아니라 계정에 남아 다른 프로젝트에도 적용되므로 이 프로젝트에만 해당하는 내용을 넣지 않습니다.

실습 3 결과 파일

C:\Users\Administrator\lab\CLAUDE.md · 코딩 컨벤션 끝에 더한 부분

새 규칙은 `@src/core/domain/rules/r001_required_fields.py`의 구조를 따릅니다.

판정 조건과 위반 문구는 규정 문서에 적힌 대로 씁니다.

`@docs/경비규정.md`

데이터 속성 이름은 ``docs/데이터-스키마.md``에 있습니다.
규칙마다 보는 문서가 아니므로 경로만 적습니다.

C:\Users\Administrator\claude\CLAUDE.md · 전역 파일 전문

답변은 한국어로 합니다.

같은 저장소의 문서인데 표기를 다르게 했습니다.

규정은 규칙을 만들 때마다 필요하므로 붙였고, 스키마는 속성 이름을 다 정한 뒤로는 가끔만 보므로 경로로 남겼습니다. 오른쪽 전역 파일은 저장소 밖에 있어 다른 프로젝트에서도 함께 적용됩니다.

파일 크기 관리

- **매 요청에 실리는 부담:** 파일이 길어질수록 남는 대화 공간이 줄고 토큰 비용이 늘어납니다. @ 로 붙인 문서의 길이도 여기에 더해집니다.
- **항목 수에 따른 누락:** 지침이 많아질수록 그중 일부만 반영되는 일이 생깁니다.
- **나누는 방법:** 자주 참고하지 않는 문서는 @ 를 떼고 경로만 남깁니다. 특정 폴더에서만 필요한 규약은 그 폴더의 CLAUDE.md 로 내립니다.

파일이 일정 크기를 넘으면 Claude Code가 경고를 표시합니다.

그때는 항목을 지우기보다 참조로 바꾸거나 폴더별로 나누는 쪽을 먼저 검토해 주세요. 지침을 지우면 그 내용을 매번 다시 설명해야 합니다.

메모리와 설정의 범위 대응

범위	메모리 파일	설정 파일	커밋
내 계정 전체	<code>~/.claude/CLAUDE.md</code>	<code>~/.claude/settings.json</code>	안 함
이 프로젝트 · 팀	<code>./CLAUDE.md</code>	<code>.claude/settings.json</code>	함
이 프로젝트 · 개인	<code>./CLAUDE.local.md</code>	<code>.claude/settings.local.json</code>	안 함

두 축이 같은 모양으로 나뉩니다.

무엇을 시킬지는 왼쪽에, 무엇을 허용할지는 오른쪽에 적습니다. 가운데 줄만 저장소에 올라가므로 팀과 나눌 내용인지가 판단 기준입니다.

권한 규칙의 범위 선택

.claude/settings.json · 커밋되어 팀 전체에 적용

```
{ "permissions": {
  "deny": ["Edit(tests/golden/**)"],
  "allow": ["Bash(uv run pytest:*)"]
} }
```

- **팀 공유로 올릴 것:** 누가 작업해도 같아야 하는 규칙. 골든 데이터 잠금이 여기에 해당합니다.
- **개인 설정에 남길 것:** 사람마다 다를 수 있는 승인 범위. 챕터 3에서 `settings.local.json` 에 쌓인 `allow` 규칙입니다.

개인 설정에 둔 잠금은 팀에 적용되지 않습니다.

새로 합류한 사람에게는 아무 제약이 없으므로, 누가 작업해도 지켜져야 하는 규칙은 팀 공유에 둡니다.

환경변수 설정

.claude/settings.json · 세션과 모든 하위 명령에 적용

```
{ "env": {
  "PYTHONUTF8": "1",
  "PYTHONIOENCODING": "utf-8"
} }
```

- **여기에 둘 것:** 사람마다 달라지면 안 되는 값. 인코딩, 로케일, 도구 경로처럼 없으면 결과가 달라지는 설정입니다.
- **여기에 두지 않을 것:** API 키, 토큰, 비밀번호. 이 파일은 커밋되므로 저장소에 그대로 남습니다.

Windows 콘솔의 기본 인코딩 때문에 한글 출력이 깨질 수 있습니다.

위 두 값을 넣어 두면 `uv run` 으로 실행하는 명령까지 UTF-8로 통일됩니다.

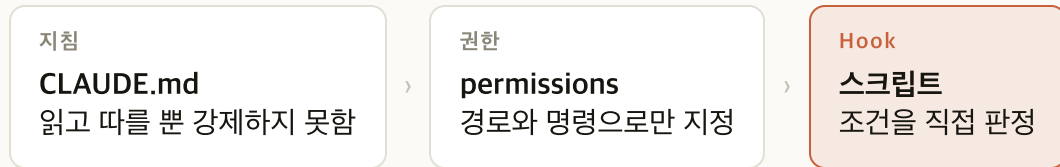
설정 항목의 배치 기준

항목	둘 곳	이유
골든 데이터 잠금	팀 공유	누가 작업해도 지켜져야 함
인코딩 환경변수	팀 공유	없으면 실행 결과가 달라짐
Hook	팀 공유	검사 기준은 사람마다 달라지면 안 됨
명령 자동 승인	개인	어디까지 맡길지는 각자의 판단
API 키	어느 쪽도 아님	비밀 정보이므로 설정 파일이 아니라 환경변수로 외부에 보관

커밋되는 파일인지를 먼저 봅니다.

팀 공유 설정은 저장소에 그대로 남아 누구나 읽습니다. 비밀 정보는 챕터 6에서 다시 다룹니다.

Hook의 필요성



- **지침으로 부족한 경우:** CLAUDE.md 에 등록하라고 적어 두어도 실제로 등록했는지는 알 수 없습니다. 확인하려면 규칙 파일 목록과 컨테이너 두 곳을 대조해야 합니다.
- **권한으로 부족한 경우:** deny 는 파일 내용을 읽고 판단하지 못합니다. 차단 대상을 경로와 명령으로만 지정하기 때문입니다.

Hook은 정해진 시점에 반드시 실행되는 스크립트입니다.
판정 기준을 코드로 적을 수 있고, 모델의 판단을 거치지 않습니다.

Hook 이벤트

도구 호출 시점

PermissionRequest	승인 프롬프트가 표시되기 전
PreToolUse	도구 실행 전. 막을 수 있음
PostToolUse	도구 성공 후
PostToolUseFailure	도구 실패 후

세션과 대화 시점

SessionStart	세션이 시작될 때
UserPromptSubmit	사용자가 입력을 보낼 때
Stop	응답이 끝날 때. 막을 수 있음
Notification	알림이 발생할 때
PreCompact · PostCompact	컨텍스트 압축 전후

이 챕터에서는 `SessionStart` 와 `PreToolUse` 두 가지를 씁니다.

`SessionStart` 는 세션이 열릴 때 작업 환경의 상태를 넣어 주고, `PreToolUse` 는 잘못된 도구 호출을 막습니다.

Hook 등록 방법

.claude/settings.json · hooks 블록

```
"hooks": {
  "PreToolUse": [
    { "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "uv run python \"${CLAUDE_PROJECT_DIR}
        /.claude/hooks/check-registration.py\""}]}
  ]
}
```

항목	정하는 것
이벤트	언제 실행할지. 앞 장의 목록에서 고름
matcher	어떤 도구에 적용할지. 생략하면 전부
command	실행할 명령. 스크립트 경로를 적음
경로	<code>\${CLAUDE_PROJECT_DIR}</code> 로 시작. 저장소 루트를 가리킴

경로는 반드시 `${CLAUDE_PROJECT_DIR}` 로 시작해야 합니다.

상대 경로로 적으면 Claude Code가 하위 폴더에서 명령을 실행하는 동안 스크립트를 찾지 못해 Hook이 실행되지 않습니다.

Hook은 설정에 등록해 둔 명령을 정해진 시점에 실행하는 장치입니다.

`settings.json` 에 이벤트와 명령을 적어 두면 그 시점마다 Claude Code가 스크립트를 실행합니다. 다음 장에서 실제 스크립트를 보고, 그 뒤에 입력과 출력의 형식을 정리합니다.

차단하는 Hook

.claude/hooks/example.py · 종료 코드로 막는 형태

```
import json, sys

data = json.load(sys.stdin)
command = data["tool_input"].get("command", "")

if "git commit" in command and has_unregistered_rule():
    # 표준 오류의 내용은 모델에 전달됨
    print("R002 가 tests/container.py 에 없습니다", file=sys.stderr)
    sys.exit(2) # 커밋 명령을 실행하지 않고 모델에 돌려보냄
```

막는 데서 끝나지 않고 무엇이 잘못되었는지 알려 줍니다.

모델이 그 문구를 읽고 스스로 고친 뒤 다시 시도합니다. 사람이 개입하지 않아도 됩니다.

Hook의 입력과 출력

등록한 스크립트가 주고받는 값

```
# 표준 입력으로 들어오는 것 (PreToolUse)
{ "session_id": "abc123",
  "tool_name": "Bash",
  "tool_input": { "command": "git commit -m ..." } }

# 돌려주는 것 1. 종료 코드. 0 통과 · 2 차단
#   표준 오류에 적은 내용이 모델에 전달됨
# 돌려주는 것 2. 표준 출력 JSON
{ "systemMessage": "사용자 화면에 표시할 문구",
  "hookSpecificOutput": { "permissionDecision": "deny" } }
```

차단만 하면 되는 검사는 종료 코드로 끝냅니다.

앞 장의 스크립트가 그 형태입니다. 표준 출력 JSON은 사용자 화면에 문구를 표시하거나 모델에 값을 실어 보낼 때 씁니다.

동적 컨텍스트 주입: 설정

.claude/settings.json · SessionStart 등록

```
"hooks": {
  "SessionStart": [
    { "hooks": [{ "type": "command",
                  "command": "uv run python \"${CLAUDE_PROJECT_DIR}/.claude/hooks/session-state.py\""} ] }
  ]
}
```

- **matcher가 없는 이유:** `matcher` 는 어떤 도구에 적용할지 고르는 값입니다. `SessionStart` 는 도구 호출과 무관하므로 적지 않습니다.
- **실행 시점:** 세션이 열릴 때 한 번 실행됩니다. 이후 대화가 이어지는 동안에는 다시 실행되지 않습니다.

파일에 적어 둘 수 없는 값을 Hook이 채웁니다.

`CLAUDE.md` 는 적어 둔 문장이 그대로 실립니다. 최근 커밋처럼 세션을 열 때마다 달라지는 값은 이 방식으로 넣습니다.

동적 컨텍스트 주입: 스크립트

.claude/hooks/session-state.py · 세션이 열릴 때 최근 커밋을 컨텍스트에 실음

```
import json, subprocess

def run(cmd):
    return subprocess.run(cmd, capture_output=True, text=True).stdout.strip()

commit = run(["git", "log", "-1", "--oneline"])
state = f"최근 커밋: {commit}"

print(json.dumps({"hookSpecificOutput": {
    "hookEventName": "SessionStart", "additionalContext": state}}))
```

세션 안에서 잘 바뀌지 않는 값은 **SessionStart** 에 둡니다.

한 번만 실리기 때문입니다. **UserPromptSubmit** 은 요청할 때마다 다시 실리므로, 매번 최신 상태여야 하는 정보에만 씁니다.

실습 4 · 환경 설정과 Hook

- 1 **상태 주입 요청:** 세션이 시작될 때 최근 커밋과 변경 파일, pytest 통과 건수를 넣어 주는 SessionStart Hook을 `${CLAUDE_PROJECT_DIR}/.claude/hooks/session-state.py` 에 Python으로 만들어 줘
- 2 **설정 정리 요청:** `.claude/settings.json` 의 env 에 PYTHONUTF8 을 1, PYTHONIOENCODING 을 utf-8 로 넣고, `settings.local.json` 의 deny 한 줄을 `settings.json` 으로 옮겨 줘
- 3 **차단 요청:** 규칙 파일이 컨테이너 두 곳에 등록되지 않았으면 git commit 을 막는 PreToolUse Hook을 `${CLAUDE_PROJECT_DIR}/.claude/hooks/check-registration.py` 에 Python으로 만들어 줘
- 4 Claude Code를 종료하고 다시 실행한 뒤 세션을 시작할 때 받은 최근 커밋과 pytest 결과를 그대로 알려 줘 로 확인
- 5 등록을 일부러 지우고 커밋을 요청해 차단되는지 확인한 뒤 되돌리기

SessionStart Hook의 출력은 터미널에 보이지 않습니다.

모델의 컨텍스트로만 들어가므로 화면에 아무것도 표시되지 않아도 실패한 것이 아닙니다.

실습 4 결과 · 설정 파일

.claude/settings.json · 환경변수, 권한, Hook을 한 파일에 모은 결과

```
{ "env": { "PYTHONUTF8": "1", "PYTHONIOENCODING": "utf-8" },
  "permissions": { "deny": ["Edit(tests/golden/**)"] },
  "hooks": {
    "SessionStart": [{ "hooks": [{ "type": "command",
      "command": "uv run python \"${CLAUDE_PROJECT_DIR}/.claude/hooks/session-state.py\""}] }],
    "PreToolUse": [{ "matcher": "Bash", "hooks": [{ "type": "command",
      "command": "uv run python \"${CLAUDE_PROJECT_DIR}/.claude/hooks/check-registration.py\""}] } ] }
```

골든 데이터 잠금을 `settings.local.json` 에서 이 파일로 옮겼습니다.

개인 설정에는 명령 자동 승인만 남습니다. 이 파일은 커밋되므로 팀 전체에 적용됩니다.

Hook 동작 결과

세션을 열 때 들어오는 내용 · additionalContext

```
최근 커밋: 63a95ec 경비 정산 시스템의 첫 규칙과 실행 경로를 만든다  
변경된 파일: src/application/container.py, tests/container.py, .claude/ 외 3개  
테스트: 2 passed in 0.02s
```

등록을 빼고 커밋을 요청한 경우 · 표준 오류로 나가는 내용

```
R002 가 tests/container.py 의 all_rules() 에 등록되지 않았습니다
```

위 세 줄은 **SessionStart** Hook이 넣은 상태 정보입니다.

아래 문구는 사람이 아니라 모델에 전달됩니다. 모델은 이를 읽고 등록을 채운 뒤 커밋을 다시 시도합니다.

Hook의 적용 기준

구분	내용
적합한 검사	판정 기준을 코드로 적을 수 있는 검사. 등록 여부, 파일 존재, 테스트 통과, 이름 규칙
적합하지 않은 검사	사람의 판단이 필요한 검사. 코드가 읽기 쉬운지, 설계가 적절한지
성능 부담	Hook 실행 시간이 그대로 대기 시간. 테스트 전체를 도구 호출마다 실행하면 작업 속도가 크게 떨어짐

Hook이 실패해도 작업은 그대로 진행됩니다.

스크립트가 오류로 종료하면 차단이 일어나지 않습니다. Hook이 동작한다고 믿고 넘어가면 실제로는 아무 검사도 이루어지지 않으므로, 만든 뒤에는 통과하는 경우와 차단되는 경우를 모두 확인해 주세요.

실습 5 · 단건 규칙 일곱 개

- 1 **일괄 요청:** 경비규정 R003 부터 R009 까지 구현해 줘. 픽스처는 `tests/rules/fixtures/` 에 있으니 그대로 써
- 2 도구 호출이 이어지는 동안 어느 파일을 만드는지 확인
- 3 `uv run pytest` 로 `9 passed` 확인
- 4 컨테이너 두 곳 등록 확인. 어긋나면 Hook이 알려 준 문구를 읽고 수정
- 5 참석인원이 빈 값일 때 R006이 규정대로 1명으로 처리하는지 확인

일곱 개를 한 번에 맡기는 것이 이 실습의 목적입니다.

절차가 파일에 있고 빠뜨린 것을 잡는 장치가 있으므로 하나씩 확인하며 진행하지 않아도 됩니다.

실습 5 진행 확인

PowerShell · uv run pytest -q

```
..... [100%]
9 passed in 0.02s
```

확인할 곳	있어야 할 것
rules/	r002 부터 r009 까지 파일 여덟 개
tests/rules/	test_r002 부터 test_r009 까지 여덟 개
container.py 두 곳	r001 부터 r009 까지 아홉 개 모두 등록

테스트 수가 규칙 수와 같아야 합니다.

9 passed 는 R001, R002와 방금 만든 일곱 개를 더한 값입니다. 모자라면 만들어지지 않은 규칙이 있는 것입니다.

다건 규칙 인터페이스

src/core/domain/rules/base.py · 챕터 3에서 이미 만든 파일

```
class RowRule(Rule):
    """단건 규칙. 청구 한 건만 보고 판정합니다."""
    def check(self, expense): ... # Violation 또는 None

class BatchRule(Rule):
    """다건 규칙. 청구 전체를 함께 보고 판정합니다."""
    def check_all(self, expenses): ... # {번호: Violation}
```

- **단건으로 판정할 수 없는 규칙:** 같은 영수증번호가 두 번 쓰였는지는 다른 건을 함께 보아야 알 수 있습니다.

서비스는 두 종류를 이미 구분해 처리합니다.

챕터 3에서 만든 `settle` 이 각 규칙의 상속 클래스를 보고 나누어 호출합니다. 규칙 파일만 추가하면 되고 서비스는 고치지 않습니다.

R010 영수증번호 중복 사용

docs/경비규정.md · 규정 원문

R010 · 영수증번호 중복 사용 (다건)

영수증번호가 입력된 건 중에서 같은 영수증번호가 2건 이상이면
해당 건을 모두 반려합니다. 하나의 영수증으로 여러 번 청구한 경우입니다.
> 같은 영수증번호로 {건수}건이 청구되었습니다

- **빈 값 처리:** 영수증번호가 없는 건은 세지 않습니다. 여럿이어도 중복이 아닙니다.
- **반려 범위:** 해당하는 건을 모두 반려합니다. 나중에 청구한 쪽만 반려하는 것이 아닙니다.

위반 문구에 건수가 들어갑니다.

판정 결과에 따라 내용이 달라지므로, 미리 상수로 두지 않고 판정할 때 만듭니다.

실습 6 · 다건 규칙 R010

- 1 규칙 생성 요청: 경비규정 R010 을 구현해 줘
- 2 상속한 클래스가 BatchRule 인지, 픽스처 r010.csv 의 어느 두 건이 중복인지 확인
- 3 uv run pytest 로 10 passed 확인
- 4 uv run settle samples\연구소.csv 로 반려 3건 확인

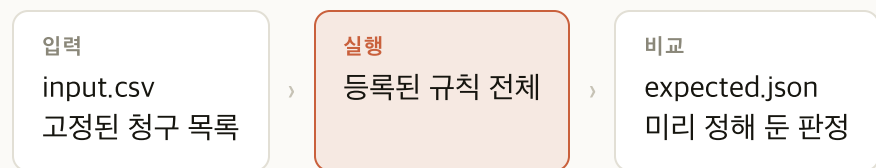
실행 결과 · samples\연구소.csv 의 반려 3건 중 R010이 잡은 2건

EXP-2026-0403	한지훈	2026-06-22	교육훈련비	210,000	반려	R010
EXP-2026-0404	서준영	2026-06-23	도서구입비	41,000	반려	R010

두 건이 함께 반려됩니다.

청구자도 계정과목도 다르지만 영수증번호가 같습니다. 한 건만 보고는 알 수 없는 위반입니다.

골든 마스터 테스트



- **규칙별 테스트가 보지 못하는 것:** 규칙 하나만 활성화해 확인하므로 규칙끼리 간섭하거나 등록이 빠진 경우를 잡지 못합니다.
- **골든이 보는 것:** 등록된 규칙 전체를 함께 실행한 결과입니다. 규칙을 하나 추가했는데 기존 판정이 달라지면 여기서 깨집니다.

챕터 3에서 **deny** 로 잠근 대상이 이 두 파일입니다.

구현이 틀렸을 때 정답을 고쳐 맞추면 안전망이 사라지므로 사람도 Agent도 수정하지 않습니다.

골든 세트의 구성

tests/golden/base/expected.json · 청구 91건의 기대 판정

```
[
  {"claim_id": "EXP-2026-0081", "is_passed": true, "violations": []},
  {"claim_id": "EXP-2026-0103", "is_passed": false, "violations": ["R005"]},
  ...
]
```

- **비교 대상:** 판정 목록만 비교합니다. 결과 객체를 통째로 비교하면 이후 챕터에서 결과에 항목이 추가될 때 골든이 깨집니다.
- **세트 단위 확장:** `base` 세트는 R001~R010을 대상으로 합니다. 챕터 5와 챕터 7에서는 세트를 하나씩 더해 이후에 만드는 규칙까지 다룹니다.

입력과 기대 출력은 저장소에 이미 있습니다.

만들 것은 두 파일을 읽어 비교하는 테스트 함수 하나입니다.

실습 7 · 골든 테스트 작성

- 1 골든 세트 base 를 내려받아 `tests\golden\base` 에 직접 배치: `tests\golden` 은 deny 규칙으로 막혀 있어 Claude Code가 만들지 못함
- 2 **테스트 작성 요청:** `tests/golden` 아래 폴더마다 `input.csv` 와 `expected.json` 을 비교하는 골든 테스트를 작성해 줘. 폴더가 늘어도 코드를 고치지 않게 해 줘
- 3 등록된 규칙 전체를 담은 `service` 픽스처를 쓰는지 확인
- 4 `uv run pytest` 로 `11 passed` 확인
- 5 규칙 하나를 컨테이너에서 잠시 빼고 골든이 깨지는지 확인한 뒤 되돌리기

5번은 안전망이 작동하는지 확인하는 절차입니다.

골든은 통과할 때가 아니라 깨질 때 의미가 있습니다. 판정이 달라졌는데도 깨지지 않으면 아무것도 지켜 주지 못합니다.

실습 7 실행 결과

PowerShell · uv run pytest

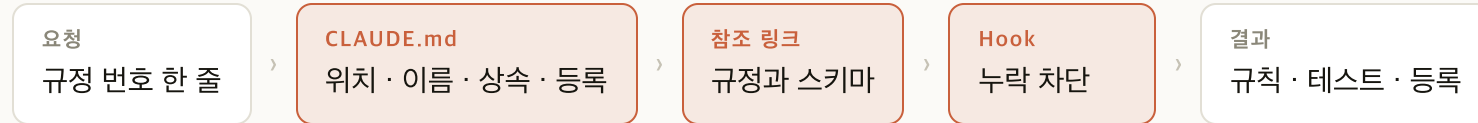
```
tests/rules/test_r001.py . [ 9%]
...
tests/rules/test_r010.py . [ 90%]
tests/test_golden.py . [100%]

===== 11 passed in 0.02s =====
```

규칙 열 개와 골든 하나로 **11 passed** 입니다.

골든이 통과했다는 것은 청구 91건의 판정이 미리 정해 둔 결과와 모두 같다는 뜻입니다.

일곱 단계의 변화



- **없앤 것:** 매번 되풀이하던 설명입니다. 절차가 사라진 것이 아니라 절차를 적는 자리가 요청에서 파일로 바뀌었습니다.
- **남은 것:** 규칙 일곱 개를 한 번에 맡길 때도 요청은 여전히 사람이 씁니다. 챕터 5에서 이 요청 자체를 명령으로 만듭니다.

이것이 별도의 맥락 설명 없이 작업을 맡기는 상태입니다.
규약이 파일에 적혀 있으므로 사람은 무엇을 할지만 말하면 됩니다.

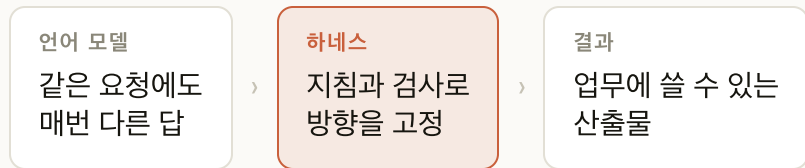
세 장치의 역할 구분

	CLAUDE.md	Skill	Hook
답는 것	프로젝트 전반의 기본 지침	반복해서 쓰는 특정 업무 절차	반드시 거쳐야 하는 검사와 게이트
컨텍스트	매 요청에 함께 실림	이름과 설명은 항상, 본문은 호출될 때만 실림	실리지 않음
동작	모델이 읽고 따름	모델이 읽고 따름	정해진 이벤트에 스크립트 실행
만드는 챕터	챕터 4	챕터 5	챕터 4

셋 중 Hook만 모델이 어길 수 없습니다.

앞의 둘은 모델이 읽고 따르는 지침입니다. Hook은 정해진 이벤트가 발생할 때 스크립트(Shell · Python)로 실행되므로 모델의 판단을 거치지 않습니다.

하네스의 역할



- **언어 모델의 역할:** 정해진 답을 꺼내 오지 않고 매번 새로 만들어 냅니다. 시키지 않은 방법까지 찾아내는 창발성도, 실행할 때마다 결과가 달라지는 불확실성도 여기서 나옵니다.
- **하네스가 주는 지침:** 계층 구조와 파일 위치, 상속할 클래스, 등록 지점 같은 개발 규약을 `CLAUDE.md` 가 정해 둡니다. Hook은 스크립트로 그 규약이 지켜졌는지 매번 같은 기준으로 검사합니다.
- **모델과 하네스의 조합:** 모델의 창발성은 그대로 두고, 하네스가 그 결과를 의도한 방향으로 이끕니다.

하네스는 저장소 안의 파일과 폴더입니다.

루트의 `CLAUDE.md` 와 `.claude/` 폴더가 여기에 해당합니다. 지금 `.claude/` 에는 설정과 Hook이 들어 있고, 챕터 5에서 Skill이 더해집니다.

확인 항목

- ☐ `CLAUDE.md` 에 네 항목 작성 완료
- ☐ 규정 문서는 `@`, 스키마 문서는 경로로 구분
- ☐ `~/.claude/CLAUDE.md` 전역 지침 작성 확인
- ☐ `.claude/settings.json` 생성과 `env` 설정 확인
- ☐ `SessionStart` Hook으로 상태 주입 확인
- ☐ 규칙 `R002` ~ `R010` 아홉 개 구현
- ☐ `tests/test_golden.py` 작성 완료
- ☐ 코딩 컨벤션에 등록 지점 두 곳 명시
- ☐ `R001` 을 본보기로 지목하는 문장 추가
- ☐ 임시 지침의 `MEMORY.md` 저장 확인
- ☐ 골든 데이터 잠금을 팀 공유로 이동
- ☐ `PreToolUse` Hook으로 차단 확인
- ☐ `R010` 의 `BatchRule` 상속 확인
- ☐ `uv run pytest` 에서 `11 passed` 확인